

Robustesse adversariale de modèles de détection d'anomalies pour les systèmes de contrôle industriel (ICS) dans un contexte de jumeau numérique

Attaques et défenses adversariales contre des détecteurs d'intrusion
Machine Learning (MLP, régression logistique, XGBoost) évalués sur
SWaT et BATADAL

Document généré à partir du code source du dépôt

`these_adv`

Version du 25 juillet 2026

Sommaire

1. Objectif et vue d'ensemble du protocole

2. Jeux de données

2.1 SWaT

2.2 BATADAL

3. Prétraitement et découpage des données

4. Modèles de détection de référence

4.1 Perceptron multicouche (MLP)

4.2 Régression logistique

4.3 XGBoost

4.4 Seuil de décision et interface commune

5. Protocole générique d'évaluation adversariale

5.1 Construction de l'ensemble d'évaluation

5.2 Taux de succès d'attaque (ASR)

5.3 Répétitions et graines aléatoires

6. Attaques en boîte blanche

6.1 Calcul du gradient selon le modèle

6.2 FGSM

6.3 PGD (multi-restart)

6.4 Carlini & Wagner (C&W)

7. Attaques en boîte noire à score (accès aux probabilités)

7.1 Square Attack

7.2 NES

8. Attaques en boîte noire à décision (accès au label seul)

8.1 HopSkipJumpAttack (HSJA)

8.2 RayS

9. Attaques par transfert (sans accès au modèle cible)

9.1 Modèles substituts

9.2 MI-FGSM

9.3 VMI-FGSM

9.4 Attaque d'ensemble (Ensemble-MI)

10. Défenses évaluées

10.1 Entraînement adversarial (MLP)

10.2 Augmentation adverse (régression logistique)

10.3 Augmentation itérative (XGBoost)

10.4 Récapitulatif des sept défenses

- 11. Réévaluation adaptative des défenses**
- 12. Métriques et indicateurs rapportés**
- 13. Reproductibilité et infrastructure logicielle**
- 14. Synthèse des hyperparamètres**

1. Objectif et vue d'ensemble du protocole

Ce protocole décrit intégralement la méthodologie expérimentale mise en œuvre pour évaluer la robustesse adversariale de modèles de détection d'anomalies/intrusions appliqués à des systèmes de contrôle industriel (ICS), dans une perspective de jumeau numérique (Digital Twin). L'ensemble de la chaîne — entraînement des modèles, génération des attaques, entraînement des défenses, réévaluation adaptative — est implémenté sous forme de scripts Python déterministes et reproductibles.

Le protocole se décompose en quatre étapes séquentielles, chacune consommant les artefacts produits par la précédente :

1. **Entraînement** de trois modèles de détection de référence (MLP, régression logistique, XGBoost) sur deux jeux de données ICS (SWaT, BATADAL), en détection binaire propre (normal / attaque).
2. **Attaque** de ces modèles selon quatre familles de menaces, correspondant à quatre niveaux d'accès décroissants de l'attaquant au modèle cible : boîte blanche (gradient exact ou approché), boîte noire à score (probabilités), boîte noire à décision (label seul), et par transfert (aucun accès, via des modèles substituts).
3. **Défense** : entraînement de sept versions durcies des modèles de référence, combinant entraînement adversarial et augmentation de données par exemples adverses, contre les familles d'attaques FGSM, PGD et Square.
4. **Réévaluation adaptative** : les modèles durcis sont ré-attaqués directement (et non simplement réévalués contre des exemples adverses précalculés sur le modèle non défendu), condition nécessaire pour mesurer une robustesse réelle plutôt qu'un artefact de généralisation.

Deux principes méthodologiques structurent l'ensemble du protocole :

- **Comparabilité** : pour un couple (dataset, modèle, graine aléatoire) donné, toutes les attaques partagent exactement le même ensemble d'évaluation, construit par un générateur pseudo-aléatoire local indépendant du reste du pipeline. Les taux de succès (ASR) sont donc directement comparables entre attaques, entre familles, et entre boîte blanche / boîte noire.
- **Reproductibilité** : toutes les sources d'aléa (NumPy, PyTorch CPU/GPU, cuDNN, mélange des DataLoader) sont fixées par une graine unique (`SEED = 42`) et ré-initialisées avant chaque étape sensible, de sorte qu'un run parallélisé (une attaque par processus) produit des résultats strictement identiques à un run séquentiel.

2. Jeux de données

Le protocole s'applique à deux jeux de données publics de sécurité ICS, choisis pour leur complémentarité : un testbed réel de traitement d'eau à forte dimensionnalité (SWaT) et un réseau de distribution d'eau simulé à faible volume mais fortement déséquilibré (BATADAL).

2.1 SWaT (Secure Water Treatment)

Propriété	Valeur
Origine	Testbed réel iTrust, Singapour
Fichier source	<code>data/raw/swat/merged.csv</code> (non versionné)
Nombre de capteurs/actionneurs (features)	51
Étapes de procédé	6 (P1–P6)
Label	colonne <code>Normal/Attack</code> — variantes textuelles unifiées ("Normal", "Attack", "Attack") → {0, 1}
Horodatage	<code>Timestamp</code> , format <code>jj/mm/aaaa hh:mm:ss AM/PM</code> , conservé aligné avec chaque split pour l'analyse temporelle

2.2 BATADAL (BATtle of the wAter DistributionAI networks)

Propriété	Valeur
Origine	Réseau simulé sous EPANET (challenge BATADAL)
Fichiers source	<code>BATADAL_dataset03.csv</code> (fonctionnement sain), <code>BATADAL_dataset04.csv</code> (scénarios d'attaque)
Nombre de features	43
Construction du jeu supervisé	Normal (0) = intégralité de <code>dataset03</code> ; Attaque (1) = uniquement les lignes de <code>dataset04</code> avec <code>ATT_FLAG == 1</code> ; les lignes <code>ATT_FLAG == -999</code> (étiquettes cachées du challenge, non fiables) sont écartées
Horodatage	<code>DATETIME</code> , format <code>jj/mm/aa hh</code>

Point de méthode commun aux deux jeux : les noms de colonnes sont nettoyés (suppression des espaces parasites, en particulier entre `dataset03` et `dataset04` pour BATADAL, dont les en-têtes diffèrent par la présence d'espaces). Ce nettoyage conditionne l'alignement correct des colonnes de features entre les deux fichiers BATADAL.

3. Prétraitement et découpage des données

Pour chaque dataset, le même pipeline unifié (`src/00-train.py`) est appliqué :

1. **Découpage stratifié en trois ensembles**, avec conservation de l'alignement des horodatages :

- Test : 20 % du total (`test_size = 0.2`), stratifié sur le label ;
- Validation : 20 % du reste train+val (`val_size = 0.2`), stratifié ;
- Train : le reliquat.

Graine de découpage : `random_state = SEED = 42`. Le jeu de test n'est **jamais** observé pendant l'entraînement (ni pour l'early stopping, ni pour la sélection d'hyperparamètres) : cette fonction est réservée au jeu de validation.

2. **Imputation des valeurs manquantes** : remplacement par la moyenne de chaque colonne, calculée **exclusivement sur le train** puis appliquée telle quelle à validation et test (évite toute fuite d'information).
3. **Standardisation** : `StandardScaler` scikit-learn, ajusté sur le train uniquement, puis appliqué (transform) à validation et test. Le scaler est sauvegardé (`scaler.pkl`) et réutilisé telle quelle pour toute inférence ultérieure, y compris pour les modèles substitués d'attaque par transfert.

Les splits (features, labels, horodatages), le scaler et les noms de colonnes sont persistés dans `artifacts/<dataset>/` et constituent la source unique consommée par tous les scripts en aval (attaques, défenses, évaluation).

4. Modèles de détection de référence

Trois familles de modèles de détection binaire (normal = 0, attaque = 1) sont entraînées indépendamment sur chaque dataset, avec pondération de classe pour compenser le déséquilibre (les attaques sont largement minoritaires).

4.1 Perceptron multicouche (MLP)

Architecture pleinement connectée : `entrée → 128 → 64 → 32 → 1` (logit), activations ReLU, dropout 0.3 après les deux premières couches cachées.

Hyperparamètre	Valeur
Fonction de perte	<code>BCEWithLogitsLoss</code> , pondérée par <code>pos_weight = n_neg / n_pos</code> (calculé sur le train)
Optimiseur	Adam, <code>lr = 1e-3</code>
Taille de batch	2048
Époques max.	50, arrêt anticipé (patience = 5) sur le F1 de validation au seuil unique
Sélection multi-graine (optionnelle)	<code>--n-seeds</code> : entraîne le MLP sur plusieurs graines (<code>SEED</code> , <code>SEED+1</code> , ...) et retient celle qui maximise le F1 de validation (utilisé pour BATADAL, 20 graines testées, meilleure retenue : seed 50)

4.2 Régression logistique

Implémentation scikit-learn (`LogisticRegression`), solveur `saga`, `C = 1.0`, `max_iter = 1000`, `class_weight = "balanced"`. Modèle linéaire : $\text{logit}(x) = w \cdot x + b$, dont le gradient par rapport à l'entrée est le vecteur de coefficients `w`, constant et exact — propriété exploitée directement par les attaques boîte blanche.

4.3 XGBoost

Hyperparamètre	Valeur
Nombre d'arbres	500 (<code>n_estimators</code>)
Profondeur max.	6
Taux d'apprentissage	0.1
Sous-échantillonnage	<code>subsample = 0.8</code> , <code>colsample_bytree = 0.8</code>
Pondération de classe	<code>scale_pos_weight = n_neg / n_pos</code>
Arrêt anticipé	<code>early_stopping_rounds = 20</code> sur le jeu de validation (logloss)

Méthode d'arbres	<code>tree_method = "hist"</code>
------------------	-----------------------------------

XGBoost étant un ensemble d'arbres constant par morceaux, son gradient exact est nul presque partout et indéfini aux nœuds de décision : la rétropropagation est impossible. Le gradient du logit est donc **approximé par différences finies centrées** (formule détaillée en §6.1), technique indispensable pour appliquer les attaques à gradient (FGSM, PGD, C&W) à ce modèle.

4.4 Seuil de décision et interface commune

Un seuil de décision **unique** est appliqué à la probabilité prédite des trois modèles, en entraînement comme en attaque : `THRESHOLD = 0.5`. Chaque modèle est enrobé d'un « wrapper » exposant une interface identique (`predict`, `predict_proba`, `logits_np`, `gradient`), ce qui permet aux scripts d'attaque de traiter MLP, régression logistique et XGBoost de façon interchangeable.

5. Protocole générique d'évaluation adversariale

5.1 Construction de l'ensemble d'évaluation

Pour un triplet (dataset, modèle, graine) donné, l'ensemble d'évaluation est construit par un générateur pseudo-aléatoire **local** (`numpy.random.default_rng(seed)`), indépendant de l'état aléatoire global, garantissant que les trois attaques boîte blanche (et, plus largement, toutes les familles) reçoivent exactement le même ensemble :

1. Tirage de `eval_nrm_size` exemples normaux (500 pour SWaT et BATADAL) parmi le jeu de test.
2. Filtrage des exemples d'attaque du jeu de test aux seuls **vrais positifs** — c'est-à-dire déjà correctement détectés par le modèle cible en clair (le principe étant de mesurer l'évasion d'attaques qui seraient sinon détectées, pas de compter les échecs de détection déjà présents en clair).
3. Tirage de `eval_atk_size` attaques parmi ces vrais positifs (500 pour SWaT ; 200 pour BATADAL, contrainte par la faible volumétrie du jeu de test, qui compte au plus ~44 attaques distinctes).

5.2 Taux de succès d'attaque (ASR)

La métrique centrale est le **taux de succès d'attaque** (Attack Success Rate) :

$$\text{ASR} = (\text{nombre de vrais positifs clean reclassés « normal » après perturbation}) / (\text{nombre total de vrais positifs clean})$$

Seuls les échantillons d'attaque déjà détectés en clair entrent dans le calcul de l'ASR — un principe appliqué uniformément à toutes les familles d'attaques et toutes les défenses. Sont également rapportés, pour chaque exécution : le F1 / précision / rappel avant et après attaque, la perturbation L_∞ (moyenne et maximale), ainsi que la marge de décision (distance logit-seuil) avant et après attaque.

5.3 Répétitions et graines aléatoires

Chaque expérience (dataset × modèle × attaque) est répétée sur **10 graines** indépendantes (`seed = 0..9`) pour les familles boîte blanche, score-based et transfert ; les statistiques rapportées sont la médiane, l'écart-type, le minimum et le maximum de l'ASR sur ces répétitions. Les scripts supportent la reprise sur interruption via des fichiers de checkpoint intermédiaires (`*_tmp.csv`), utile pour les attaques coûteuses (decision-based).

6. Attaques en boîte blanche

L'attaquant dispose d'un accès complet au modèle cible (architecture, poids, gradient exact ou approché). Trois attaques classiques de la littérature sont implémentées, sous contrainte de perturbation L_∞ bornée par $\varepsilon \in \{0.1, 0.3\}$ (les résultats rapportés dans ce protocole utilisent principalement $\varepsilon = 0.1$, $\varepsilon = 0.3$ étant réservé à l'entraînement des défenses, §10).

6.1 Calcul du gradient selon le modèle

Modèle	Nature du gradient	Méthode
MLP	Exact	Rétropropagation PyTorch (<code>autograd</code>)
Régression logistique	Exact	Formule fermée : $\partial \text{logit} / \partial x = w$ (constant)
XGBoost	Approché	Différences finies centrées, votées sur plusieurs pas

Pour XGBoost, le gradient du logit est estimé par différences finies centrées d'ordre 2 :

$$\partial g / \partial x_j \approx [g(x + \varepsilon_{fd} \cdot e_j) - g(x - \varepsilon_{fd} \cdot e_j)] / (2 \cdot \varepsilon_{fd})$$

avec $g = \log(p/(1-p))$ le logit. Coût : $2 \cdot D$ passes forward par batch (D = nombre de features). Pour stabiliser l'estimation d'une fonction en escalier bruitée (propre aux ensembles d'arbres), un **vote multi-pas** est employé : le gradient numérique est calculé pour $\varepsilon_{fd} \in \{5 \cdot 10^{-4}, 10^{-3}, 2 \cdot 10^{-3}, 5 \cdot 10^{-3}\}$, la magnitude retenue est la moyenne des quatre estimations et la direction (signe) résulte d'un vote majoritaire entre elles. La direction finale FGSM/PGD combine ce signe avec le facteur de direction de la BCE, $\text{sign}(p - y)$, pour s'assurer que la perturbation fait effectivement **baisser** le logit d'un exemple d'attaque ($y = 1$).

6.2 FGSM (Fast Gradient Sign Method)

$$x_{adv} = \text{clip}_{[x-\varepsilon, x+\varepsilon]}(x + \varepsilon \cdot \text{sign}(\nabla_x L))$$

Attaque à une seule itération, appliquée séparément selon le modèle (gradient exact pour MLP/LogReg via `BCEWithLogitsLoss`, gradient voté pour XGBoost).

6.3 PGD (Projected Gradient Descent), multi-restart

Hyperparamètre	Valeur
Itérations	200 (mode standard) / 50 (mode <code>--fast</code>)
Redémarrages (restarts)	10 (standard) / 3 (<code>--fast</code>)

Pas α	$\varepsilon / 4$ (PGD_ALPHA_K = 4)
--------------	---------------------------------------

À chaque redémarrage, un bruit uniforme initial $U(-\varepsilon, \varepsilon)$ est ajouté, puis l'exemple est itérativement mis à jour dans la direction du signe du gradient et projeté sur la boule L_∞ centrée sur l'original :

$$x_{t+1} = \text{clip}_{[x-\varepsilon, x+\varepsilon]}(x_t + \alpha \cdot \text{sign}(\nabla_x L(x_t)))$$

Le meilleur résultat (logit final le plus bas, donc l'évasion la plus forte) est conservé à travers les redémarrages — une vraie compétition entre restarts (initialisation à $+\infty$). Cette formulation multi-restart, la plus exigeante pour le défenseur, sert de référence « pire cas » en boîte blanche (à la Madry et al.).

6.4 Carlini & Wagner (C&W)

Modèle	Méthode d'optimisation	Itérations	Taux d'apprentissage
MLP	Reparamétrage $\tanh$ + Adam, warm-start PGD (20 iters $\times$ 3 restarts)	300 (standard) / 150 (--fast)	0.005
LogReg	Adam manuel sur les logits, clip L_∞	500 / 150	0.01
XGBoost	Adam manuel, warm-start PGD, gradient par différences finies	500 / 150	0.005

Fonction objectif minimisée (variante Carlini & Wagner à marge, formulation « f6 ») :

$$L = \max(0, \text{logit}(x_{\text{adv}}) - \text{seuil}_{\text{logit}} + \kappa)$$

avec $\kappa = 0$ (paramètre de confiance) et $\text{seuil}_{\text{logit}} = \log(\text{THRESHOLD} / (1 - \text{THRESHOLD})) = 0$. Pour le MLP, la perturbation est reparamétrée via $\delta = \varepsilon \cdot \tanh(w)$ afin de garantir intrinsèquement la contrainte L_∞ , w étant optimisé librement par Adam ; l'initialisation de w est un « warm-start » depuis un PGD court, ce qui accélère et stabilise la convergence.

7. Attaques en boîte noire à score

L'attaquant n'observe que les probabilités prédites par le modèle (`predict_proba`), sans accès au gradient. Toutes les requêtes sont batchées pour limiter le coût.

7.1 Square Attack

Hyperparamètre	Valeur
Requêtes max.	2000
Fraction initiale de features perturbées (<code>p_init</code>)	0.3, décroissante selon $p(q) = \max(p_init \cdot (1 - q/\max_queries)^{0.5}, 0.05)$

À chaque requête, un carré de $\max(p(q) \cdot D, 1)$ features est sélectionné aléatoirement et perturbé de $\pm \epsilon$ (recherche aléatoire par carrés, sans gradient) ; la perturbation candidate est retenue seulement si elle augmente la perte BCE par rapport à l'entropie croisée courante. Arrêt anticipé dès que 100 % des exemples actifs sont évadés.

7.2 NES (Natural Evolution Strategies)

Hyperparamètre	Valeur
Itérations	100
Échantillons antithétiques par itération	50 (<code>n_samples</code>)
Écart-type du bruit (σ)	0.01
Taux d'apprentissage	0.01

Estimation du gradient par différences finies aléatoires antithétiques (paires $+\sigma \cdot u$ / $-\sigma \cdot u$) :

$$\nabla_x L \approx (1/n) \sum [(L(x+\sigma u) - L(x-\sigma u)) / (2\sigma)] \cdot u$$

suivie d'une mise à jour au signe du gradient estimé, projetée sur la boule L_∞ .

8. Attaques en boîte noire à décision

L'attaquant n'observe que le label prédit (0/1), sans accès aux probabilités ni au gradient — le scénario le plus contraint. Ces attaques sont les plus coûteuses en requêtes et sont, à ce titre, sous-échantillonnées (au plus 300 attaques par (seed, modèle), 100 en mode `--fast`) et checkpointées.

8.1 HopSkipJumpAttack (HSJA)

Hyperparamètre	Valeur (standard / fast)
Itérations principales	40 / 10
Estimations de gradient par itération (<code>n_est</code>)	100 / 30
Pas initial	0.1, décroissant en $1/\sqrt{(it+1)}$
Tentatives d'initialisation	200
Pas de recherche binaire	15

Principe : (1) initialisation par tirage aléatoire uniforme dans $[-\epsilon, \epsilon]$ jusqu'à obtenir un point adverse, affiné par recherche binaire vers la frontière de décision ; (2) à chaque itération, estimation de la direction du gradient de la frontière par échantillonnage aléatoire de directions unitaires u et vote de signe selon que $x + 0.01 \cdot u$ reste ou non adverse ; (3) pas dans cette direction estimée, projeté sur la boule L_∞ , puis nouvelle recherche binaire si le point reste adverse et rapproche la distance de l'original. Implémentation entièrement batchée (tous les exemples actifs sont traités en un seul appel `predict` par étape), correction majeure de performance par rapport à une implémentation naïve exemple-par-exemple.

8.2 RayS

Hyperparamètre	Valeur (standard / fast)
Itérations	60 / 15
Pas de recherche d'amplitude	15 / 10
Tentatives d'initialisation	200

Attaque par recherche de rayon discret : une direction candidate $d \in \{-1, +1\}^D$ est initialisée aléatoirement, puis à chaque itération un bit (une coordonnée) de d est retourné aléatoirement ; si la mutation reste adverse à amplitude ϵ , une recherche binaire minimise l'amplitude effective de la perturbation le long de cette direction ; sinon, la direction précédente est réessayée. Le résultat est la perturbation de plus faible norme L_∞ efficace trouvée parmi les directions explorées.

9. Attaques par transfert

L'attaquant n'a **aucun** accès au modèle cible (ni gradient, ni score, ni label) : il entraîne des modèles substituts sur des données représentatives, génère des exemples adverses sur ces substituts par des attaques à gradient, puis transfère ces perturbations vers la cible sans jamais l'interroger.

9.1 Modèles substituts

Trois architectures MLP de complexité différente sont entraînées comme substituts indépendants (diversité architecturale favorisant la transférabilité) :

Substitut	Architecture
Sub1 (MLP)	entrée → 128 → 64 → 32 → 1 (identique au modèle cible)
Sub2 (SmallMLP)	entrée → 64 → 32 → 1
Sub3 (DeepMLP)	entrée → 256 → 128 → 64 → 32 → 1

Entraînement : Adam ($lr = 1e-3$), `BCEWithLogitsLoss` pondérée, 30 époques max., arrêt anticipé (patience 5) sur le F1 du **vrai** jeu de validation. Un bruit gaussien ($\sigma = 0.02$) est ajouté aux entrées à l'entraînement pour diversifier les substituts d'une graine à l'autre et améliorer la transférabilité. Sur SWaT (~1 M lignes), l'ensemble d'entraînement des substituts est sous-échantillonné à 100 000 lignes maximum (tirage uniforme préservant le ratio de classes) — un attaquant réel n'ayant de toute façon jamais accès au jeu d'entraînement complet de la victime.

9.2 MI-FGSM (Momentum Iterative FGSM)

Hyperparamètre	Valeur
Itérations	40
Pas α	2ε / itérations
Coefficient de momentum μ	1.0

$$g_{t+1} = \mu \cdot g_t + \nabla_x L(x_t) / \|\nabla_x L(x_t)\|_1 \quad ; \quad x_{t+1} = \text{clip}_{[x-\varepsilon, x+\varepsilon]}(x_t + \alpha \cdot \text{sign}(g_{t+1}))$$

Le gradient est normalisé en norme L1 avant accumulation du momentum, technique connue pour améliorer la transférabilité inter-modèles par rapport à un PGD/FGSM itératif classique.

9.3 VMI-FGSM (Variance-tuned MI-FGSM)

Hyperparamètre	Valeur
----------------	--------

Itérations	40
μ	1.0
β (rayon de voisinage / correction de variance)	0.3
Voisins échantillonnés par itération	10

Extension de MI-FGSM : à chaque itération, le gradient est corrigé par la variance locale, estimée en moyennant le gradient sur 10 points voisins tirés uniformément dans $[-\beta\epsilon, \beta\epsilon]$ autour du point courant :

$$g_{\text{used}} = \nabla_x L(x_t) + \beta \cdot (\nabla_x L(x_t) - \text{moyenne}_{\text{voisins}} \nabla_x L)$$

puis normalisation L1 et accumulation de momentum identique à MI-FGSM. Objectif : réduire le risque de convergence vers un optimum local propre au substitut, non partagé par la cible.

9.4 Attaque d'ensemble (Ensemble-MI)

Variante de MI-FGSM agrégeant les gradients (normalisés L1) des trois substituts à poids égaux (1/3 chacun) avant application du momentum — hypothèse qu'une perturbation efficace contre plusieurs architectures substituts simultanément transfère mieux vers un modèle cible inconnu.

9.5 Évaluation du transfert

L'ASR de transfert est calculé en appliquant à la **victime** les exemples adverses générés sur le **substitut** (celui-ci n'étant jamais requêté davantage) ; les substituts sont ré-entraînés indépendamment à chaque graine, mais partagés entre les modèles cibles au sein d'une même graine (le transfert ne dépendant pas de la victime).

10. Défenses évaluées

Sept versions durcies des trois modèles de référence sont produites, combinant entraînement adversarial en ligne (MLP) et augmentation de données par exemples adverses (les trois modèles), contre les attaques FGSM, PGD (boîte blanche) et Square (boîte noire à score). **Choix de conception assumé** : les défenses sont entraînées à $\epsilon_{AT} = 0.3$ alors que les attaques d'évaluation utilisent $\epsilon = 0.1$ — un modèle durci contre des perturbations plus larges généralisant mieux vers des perturbations plus petites.

10.1 Entraînement adversarial (MLP) — AT-FGSM, AT-PGD

Schéma à la Madry et al., en ligne, par batch : à chaque batch d'entraînement, (1) génération d'exemples adverses sur le modèle **courant** (FGSM ou PGD à 7 itérations / 1 restart), (2) mélange d'une fraction `mix_ratio = 0.5` du batch par ses versions adverses, (3) rétropropagation sur le batch mixte (mode `train`, dropout actif). Arrêt anticipé (patience 7) sur le F1 de validation propre. 50 époques max., `lr = 1e-3`, batch 2048.

10.2 Augmentation itérative black-box (MLP) — AT-Square

Square étant une attaque à requêtes (non différentiable, coûteuse à régénérer à chaque batch), la défense correspondante est **hors-ligne** : schéma « génère → accumule → réentraîne » sur 5 rounds, 2000 requêtes Square par round (budget aligné sur celui utilisé en évaluation). À chaque round, le modèle de la génération précédente attaque le jeu d'entraînement (vrais positifs), les exemples adverses générés sont ajoutés au jeu cumulé, et un MLP neuf est réentraîné dessus. Le round final retenu n'est ni le premier ni systématiquement le dernier (le F1 de validation peut se dégrader fortement avec l'accumulation), mais celui qui maximise un score combiné $F1_{val} - ASR_{val}(FGSM, \epsilon=0.1)$ — un critère purement F1 favoriserait le round le moins durci, donc le moins robuste.

10.3 Augmentation adverse (régression logistique) — Aug-FGSM, Aug-Square

Génération d'exemples adverses (FGSM ou Square) sur le jeu d'entraînement complet, ajout au jeu d'entraînement, puis réentraînement. Une **recherche sur grille** du paramètre de régularisation $C \in \{1, 0.3, 0.1, 0.03, 0.01, 0.003, 0.001, 3 \cdot 10^{-4}, 10^{-4}\}$ sélectionne, sur le jeu de validation, le candidat maximisant $F1_{val} - ASR_{val}(FGSM, \epsilon=0.1)$ parmi les candidats non dégénérés ($F1 \geq 30\%$ du F1 baseline). Cette recherche corrige un défaut identifié empiriquement : à $C = 1.0$ fixe, le refit sur des points quasi identiques au normal mais étiquetés attaque fait exploser la norme des coefficients, ce qui — pour un modèle linéaire dont la sensibilité L_∞ est gouvernée par le ratio $marge/\|w\|_1$ — **dégrade** la robustesse au lieu de l'améliorer.

10.4 Augmentation itérative (XGBoost) — Aug-FGSM-Iter, Aug-Square-Iter

Même schéma « génère → accumule → réentraîne » que pour le MLP black-box, mais appliqué de façon générique aux attaques FGSM (gradient par différences finies) et Square (requêtes) : 3 rounds, chaque round produisant un XGBoost neuf (mêmes hyperparamètres qu'en §4.3, arrêt anticipé sur le jeu de validation propre) entraîné sur le jeu cumulé.

10.5 Récapitulatif des sept défenses

#	Modèle de base	Défense	Attaque d'entraînement	Fichier produit
1	MLP	AT-FGSM	FGSM (blanche, en ligne)	<code>mlp_at_fgsm.pt</code>
2	MLP	AT-PGD	PGD (blanche, en ligne)	<code>mlp_at_pgd.pt</code>
3	MLP	AT-Square	Square (noire à score, hors-ligne itérative)	<code>mlp_at_square.pt</code>
4	LogReg	Aug-FGSM	FGSM (blanche) + recherche de C	<code>logreg_aug_fgsm.pkl</code>
5	LogReg	Aug-Square	Square (noire à score) + recherche de C	<code>logreg_aug_square.pkl</code>
6	XGBoost	Aug-FGSM-Iter	FGSM (blanche, gradient FD) itérative, 3 rounds	<code>xgb_iter_fgsm_r3.json</code>
7	XGBoost	Aug-Square-Iter	Square (noire à score) itérative, 3 rounds	<code>xgb_iter_square_r3.json</code>

11. Réévaluation adaptative des défenses

Point méthodologique central : les modèles durcis ne sont pas évalués contre des exemples adverses précalculés sur le modèle non défendu (ce qui surestimerait trivialement la robustesse), mais **ré-attaqués directement** — chaque attaque (blanche comme noire) est régénérée en interrogeant explicitement le modèle défendu. C'est le standard académique d'évaluation dit « adaptatif ».

- **Réévaluation boîte blanche** (`evaluate.py`) : les quatre défenses entraînées contre une attaque à gradient (AT-FGSM, AT-PGD, Aug-FGSM, Aug-FGSM-Iter) sont ré-attaquées par FGSM, PGD et C&W, à $\epsilon = 0.1$, sur 3 graines. L'ASR obtenu est comparé à la médiane de l'ASR baseline (modèle non défendu) issue des JSON produits par le run boîte blanche initial.
- **Réévaluation boîte noire** (`evaluate_blackbox.py`) : les sept défenses (y compris les trois variantes durcies contre Square) sont ré-attaquées par les trois familles boîte noire complètes (Square, NES, HSJA, RayS, MI-FGSM, VMI-FGSM, Ensemble-MI), ce qui permet de mesurer si un durcissement entraîné contre une attaque particulière (p. ex. gradient exact) généralise à une autre catégorie d'attaque (p. ex. à requêtes, ou par transfert) — et réciproquement.

Les substituts utilisés dans la réévaluation de transfert sont entraînés uniquement sur des données **propres** (jamais sur le modèle défendu), conformément à l'hypothèse de menace par transfert (l'attaquant n'observe jamais la cible, défendue ou non).

12. Métriques et indicateurs rapportés

Indicateur	Définition
ASR	Fraction des vrais positifs clean reclassés « normal » après perturbation (§5.2)
F1 / précision / rappel (clean)	Sur le jeu d'évaluation non perturbé, au seuil unique 0.5
F1 / précision / rappel (adversarial)	Sur le jeu d'évaluation après remplacement des vrais positifs par leur version perturbée
L_∞ (moyen, max)	Amplitude maximale de la perturbation par feature, par échantillon, agrégée sur l'ensemble d'évaluation
Marge (clean, adverse, médiane, min)	Distance logit – seuil _{logit} ; une marge négative signale une évacion
Chute de marge moyenne	$\text{marge}_{\text{clean}} - \text{marge}_{\text{adv}}$, moyenne sur les vrais positifs
ΔASR (défenses)	$\text{ASR}_{\text{défendu}} - \text{ASR}_{\text{baseline}}$, médiane sur les graines

Pour l'analyse temporelle (alignement avec les horodatages du procédé industriel), un sous-échantillon par-échantillon (50 exemples max. par (seed, modèle, attaque), tirage reproductible) est exporté avec succès/échec, marge adverse, L_∞ et horodatage d'origine.

13. Reproductibilité et infrastructure logicielle

13.1 Contrôle de l'aléa

Une fonction unique `set_all_seeds(seed)` fixe séquentiellement : le générateur Python natif (`random`), NumPy, PyTorch CPU et CUDA, et force le mode déterministe de cuDNN (`deterministic = True`, `benchmark = False`). Cette fonction est ré-appelée avant chaque graine et avant chaque attaque au sein d'une même graine, garantissant qu'un run parallélisé (un processus par attaque ou par famille) reproduit exactement les résultats d'un run séquentiel `--attack all`.

13.2 Environnements logiciels

Deux environnements séparés : un environnement « entraînement / attaques / défenses » (PyTorch, XGBoost, scikit-learn) et un environnement « démo » minimal (Streamlit, lecture seule des résultats déjà produits, aucun recalcul).

Composant	Version
Python (scikit-learn)	1.8.0
XGBoost	3.2.0
NumPy	2.4.4
Pandas	3.0.2
Matplotlib	3.10.9
Streamlit (démo)	1.56.0

13.3 Organisation du code et des artefacts

```
these_adv/
├─ data/raw/<dataset>/      données brutes (non versionnées)
├─ src/
│   ├── 00-train.py          entraînement des 3 modèles de référence
│   ├── common.py            constantes, chargement, ASR, seeds
│   ├── models.py            architectures + wrappers (interface commune)
│   ├── whitebox.py / whitebox_run.py  FGSM / PGD / C&W
│   ├── blackbox.py / blackbox_run.py  Square / NES / HSJA / RayS
│   ├── transfer.py          substituts, MI-FGSM, VMI-FGSM
│   ├── defenses.py          entraînement des 7 modèles durcis
│   ├── evaluate.py          réévaluation adaptative (boîte blanche)
│   ├── evaluate_blackbox.py  réévaluation adaptative (boîte noire)
│   └─ plot_results.py       génération des figures
└─ artifacts/<dataset>/     modèles entraînés, splits, scaler
```

— results/<dataset>/	métriques (csv/json) + figures
— demo/	interface Streamlit de restitution

13.4 Commandes du pipeline complet

```
python src/00-train.py          --dataset swat
python src/whitebox_run.py      --dataset swat --eps 0.1 --attack all
python src/blackbox_run.py      --dataset swat --eps 0.1
python src/defenses.py          --dataset swat
python src/evaluate.py          --dataset swat --eps 0.1
python src/evaluate_blackbox.py --dataset swat --eps 0.1
python src/plot_results.py      --dataset swat --eps 0.1
```

14. Synthèse des hyperparamètres

14.1 Attaques

Famille	Attaque	Accès requis	Hyperparamètres clés
Boîte blanche	FGSM	Gradient	1 itération, $\epsilon \in \{0.1, 0.3\}$
	PGD	Gradient	200 iters $\times$ 10 restarts, $\alpha = \epsilon/4$
	C&W	Gradient	300–500 iters, Adam, warm-start PGD, $\kappa = 0$
Boîte noire — score	Square	Probabilités	2000 requêtes, $p_{\text{init}} = 0.3$
	NES	Probabilités	100 iters, 50 échantillons/iter, $\sigma = 0.01$
Boîte noire — décision	HSJA	Label	40 iters, 100 estim./iter, recherche binaire 15 pas
	RayS	Label	60 iters, recherche d'amplitude 15 pas
Transfert	MI-FGSM	Aucun (substitut)	40 iters, $\mu = 1.0$, norme L1
	VMI-FGSM	Aucun (substitut)	40 iters, $\mu = 1.0$, $\beta = 0.3$, 10 voisins
	Ensemble-MI	Aucun (3 substituts)	MI-FGSM agrégé, poids égaux 1/3

14.2 Modèles et défenses

Modèle	Architecture / algorithme	Défenses associées
MLP	128-64-32-1, dropout 0.3, Adam	AT-FGSM, AT-PGD (en ligne) ; AT-Square (itérative, 5 rounds)
Régression logistique	saga, C recherché sur grille, class_weight balanced	Aug-FGSM, Aug-Square (augmentation + recherche de C)
XGBoost	500 arbres, profondeur 6, lr 0.1, gradient par différences finies	Aug-FGSM-Iter, Aug-Square-Iter (itérative, 3 rounds)

Document généré automatiquement à partir du code source du dépôt de recherche (répertoire [src/](#)), en cohérence avec le [README.md](#) et le [GUIDE_CODE.md](#) du projet. Toute modification des scripts sources doit être répercutée dans ce protocole avant citation dans le mémoire.